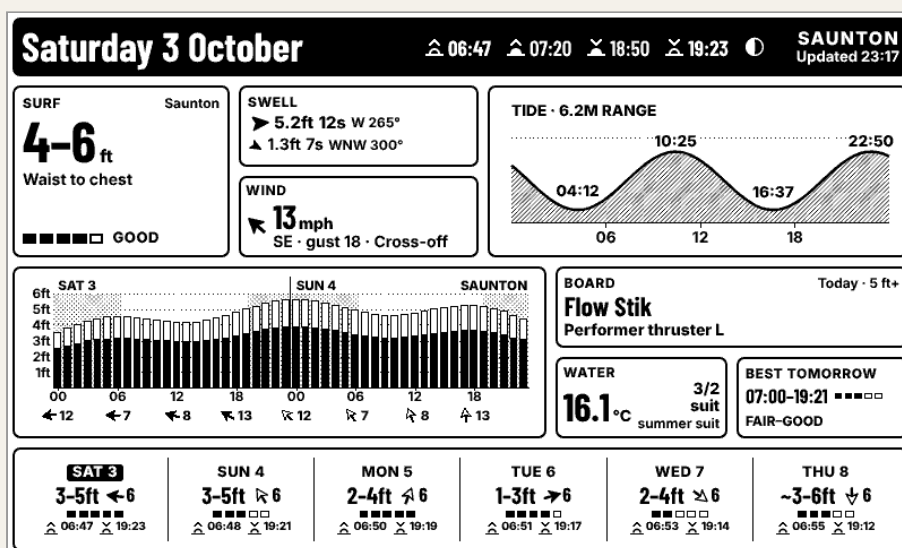


Build your own e-ink surf frame

A framed e-ink screen that shows the surf forecast for your beaches and updates itself every three hours. It's built from an off-the-shelf DIY kit and free cloud services, with nothing left switched on at home.



CODE

github.com/callumohare/eink-surf-frame

TIME

3-4 hours + building the frame

RUNNING COST

£0 (free tiers)

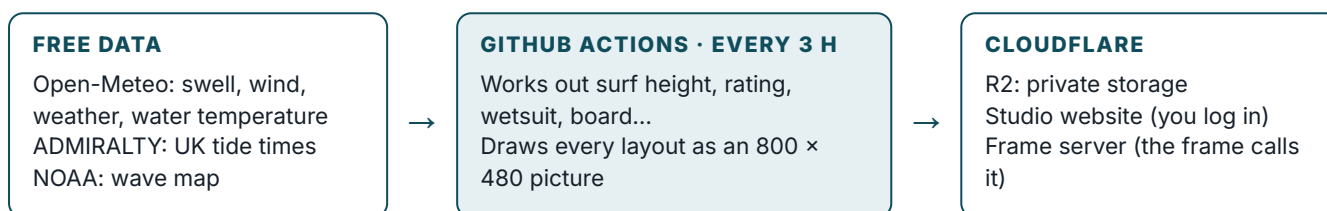
Before you start

This guide takes you from nothing to a working frame on your desk. It assumes no coding experience: you'll copy commands into a terminal window and click through a few websites. Work through the Parts in order. Each one ends at a safe place to stop and come back another day.

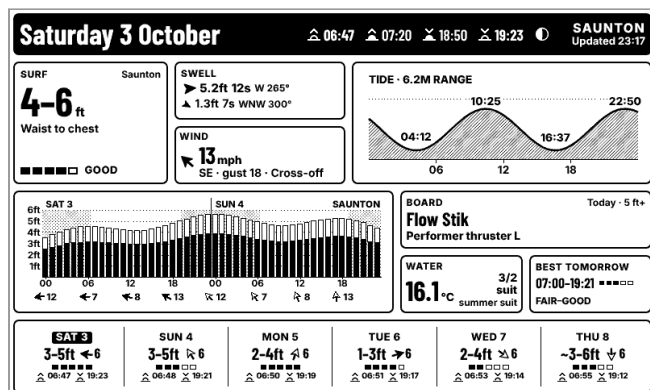
What you're building

Every three hours a computer in the cloud (GitHub Actions) downloads the forecast for your beaches, draws one picture for each of your layouts, and saves the pictures in private online storage (Cloudflare R2). Two small websites (Cloudflare Workers) sit on top of that storage:

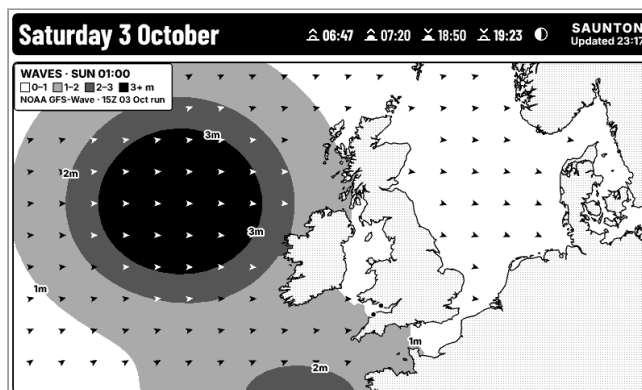
- the **studio**, where you log in to choose and design what the frame shows, and
- the **frame server**, which the frame wakes up to call, downloads its picture from, and then goes back to sleep.



Your computer is only needed during setup, and later if you want to design layouts offline. The frame runs on its own battery for weeks.



The "Classic" layout (sample data)



The "Wave map" layout: press the button on the frame to switch to it

Contents

1	Order the hardware	10 min	10	First cloud render	20 min
2	Install the tools	30 min	11	On-time updates and hands-off deploys	25 min
3	Get your own copy of the code	20 min	12	Set up the frame	60 min
4	Try it on your computer	15 min	13	Build it into a picture frame	1–2 h
5	Make it yours: beaches and settings	30 min	14	Using the studio and the button	
6	Cloudflare account and storage	20 min	A	Optional extras	
7	Put the two websites online	20 min	B	Troubleshooting	
8	Lock the studio so only you can open it	25 min	C	Upkeep, costs and security	
9	UK tide times from ADMIRALTY	20 min	D	Living outside the UK?	

What you need

- ☐ **The hardware** (Part 1): the Sseed TRMNL 7.5" (OG) DIY Kit, a **data** USB-C cable, and a picture frame.
- ☐ **A Mac or Windows computer** you can install software on (not a locked-down work laptop). Chrome or Edge for flashing the frame.
- ☐ **Your phone**, for two-factor sign-in codes and for setting up the frame's Wi-Fi.
- ☐ **Access to your router's settings**, to make a guest Wi-Fi network for the frame.
- ☐ **A password manager** (1Password, Bitwarden, Apple Passwords or similar) for the keys you'll collect.
- ☐ About **3–4 hours**, split however you like, plus time to build the frame.

Accounts you'll create (all free)

ACCOUNT	WHAT IT DOES	PART
GitHub	Stores your copy of the code and runs the 3-hourly job	3
Cloudflare	Private storage, the studio website and the frame server. Asks for a payment card to switch on storage, but this project stays far inside the free allowance.	6
ADMIRALTY Developer Portal	Official UK tide times (free "Discovery" plan, renewed yearly)	9

Words you'll meet

WORD	PLAIN MEANING
Terminal	A window where you type commands instead of clicking. Part 2 shows you how to open it.
Command	One line you paste into the terminal, then press Enter. Copy them exactly from this guide.
Repository (repo)	A project folder stored on GitHub, with a history of every change.
Commit / Push	Commit = save a snapshot of your changes. Push = upload that snapshot to GitHub.
Secret, key, token	A password-like value. Anyone who has it can use that account, so it never goes into a file, a chat or a screenshot.
Worker	A small website that runs on Cloudflare's servers.
Bucket	A private online folder in Cloudflare R2.
Deploy	Upload a Worker so it goes live on the internet.

Your secrets record

Create one secure note in your password manager called **Surf Frame** and copy these headings into it now. Fill each one in as you reach it.

ITEM	PART	ITEM	PART
Cloudflare Account ID	6	Access AUD tag	8
R2 Access Key ID	6	ADMIRALTY subscription key	9
R2 Secret Access Key	6	GitHub render token	11
Device token	7	Cloudflare deploy token	11
Frame server address	7	Guest Wi-Fi name and password	12
Studio address	7	The frame's MAC address	12

GOLDEN RULE

Never paste a key into an email, a chat, a forum post, a screenshot, or any file inside the project folder. The project keeps them in two safe places only: GitHub's encrypted secrets and Cloudflare's encrypted secrets.

1 Order the hardware

10 min (plus delivery)

Order these first: everything up to Part 11 works without the frame, so you can do the cloud setup while you wait for delivery.

The kit

Seeed Studio TRMNL 7.5" (OG) DIY Kit. It's sold by Seeed Studio (seeedstudio.com; search "TRMNL 7.5 Inch OG DIY Kit") and by electronics resellers. The kit contains:

- the XIAO ePaper driver board (an ESP32-S3 with Wi-Fi, battery charging and three buttons),
- a 7.5" black-and-white e-ink panel (800 × 480 pixels; this project uses it with 4 shades of grey),
- a ribbon extension cable with adapter, a 2000 mAh battery and a Wi-Fi antenna.

You won't need a TRMNL account or subscription: the frame runs TRMNL's free, open-source firmware pointed at your own server.

Also get

- **A USB-C data cable.** Many cables only charge. If your computer doesn't see the board later, the cable is the first suspect.
- **A picture frame** with a mount (passe-partout). A **20 × 15 cm (8 × 6 in)** frame works well, with a mount whose window is about **162 × 97 mm**: the screen's picture area is 163.2 × 97.7 mm, so the mount just covers its edge. Framing shops cut custom mounts cheaply. Choose a frame with some depth behind the glass: the board, battery and cable sit on the back of the backing board.
- Thin tape (Kapton or low-tack), small self-adhesive foam pads, and a craft knife for a cable slot.

TIP

Measure your panel and mount before cutting anything. If the mount ends up covering a few pixels at the edge, the studio has an **Edge margin** setting that shrinks the picture to fit (Part 14).

2 Install the tools

30 min

You'll install four free programs: Python (runs the forecast code), Node.js (uploads the websites to Cloudflare), Visual Studio Code (a safe editor for the settings files) and GitHub Desktop (Part 3). Follow the column for your computer only.

2.1 Open a terminal and learn the three rules

- **Mac:** press `Cmd` + `Space`, type `Terminal`, press Enter. A window opens with a line ending in `%` or `$`.
- **Windows:** press the Windows key, type `cmd`, click **Command Prompt**. A black window opens with a line ending in `>`. Use Command Prompt (not PowerShell) for everything in this guide.

THE THREE RULES

- 1 Copy a command from the grey boxes, paste it into the terminal (Mac: `Cmd` + `V`; Windows: right-click), then press Enter.
- 2 Wait until the `%`, `$` or `>` line comes back before typing the next command. Some take a few minutes.
- 3 Red text or the word **error** means stop. Check Appendix B before carrying on.

2.2 Install Python (3.12 or newer)

MAC

- 1 Go to **python.org/downloads** and click the yellow **Download Python 3.x** button.
- 2 Open the downloaded `.pkg` and click Continue / Agree / Install until it finishes.
- 3 In the Finder window that opens, double-click **Install Certificates.command**.
- 4 Quit Terminal (`Cmd` + `Q`), open it again and check:

```
python3 --version
```

On a Mac, type `python3`, not `python`, until Part 4 says otherwise.

You should see `Python 3.12` or higher.

WINDOWS

- 1 Go to **python.org/downloads** and click **Download Python 3.x**.
- 2 Open the file. On the first screen tick **"Add python.exe to PATH"** at the bottom (this matters), then **Install Now**.
- 3 Close Command Prompt, open it again and check:

```
python --version
```

On Windows always type `python`. Where this guide says `python3`, type `python`.

2.3 Install Node.js

Go to **nodejs.org**, download the version labelled **LTS**, and install it with every default. Close and reopen the terminal, then check both of these print a version number:

```
node --version
npm --version
```

2.4 Install Visual Studio Code

Download it from **code.visualstudio.com** and install with the defaults. Use it for every settings file. Don't use TextEdit, Notepad or Word: they can swap straight quotes `"` for curly ones and break the files.

STOP POINT You have all the tools. Nothing is online yet.

3 Get your own copy of the code

20 min

You'll make your own **private** copy of the project on GitHub, then download it to your computer. Private matters: your copy will hold your settings and the job's logs.

3.1 Create a GitHub account and lock it down

- 1 Go to github.com/signup, create a free account and confirm your email.
- 2 Turn on two-factor sign-in: profile picture (top right) → **Settings** → **Password and authentication** → **Enable two-factor authentication**. Choose an authenticator app, scan the QR code and type the 6-digit code.
- 3 Save the recovery codes GitHub shows in your password manager.

3.2 Make your private copy

- 1 Open github.com/callumohare/eink-surf-frame.
- 2 Click the green **Use this template** button → **Create a new repository**.
- 3 Repository name: `surf-frame`. Choose **Private**. Click **Create repository**.

WHY "USE THIS TEMPLATE" AND NOT "FORK"?

A fork of a public project has to stay public. A copy made from a template is a brand-new repository that can be private and has no link back to the original.

Nothing runs in your copy yet. The 3-hourly job and the website updates stay switched off until you create a switch called `SURF_FRAME_READY` in Part 10, so it won't fail and email you while you're setting up.

3.3 Download it with GitHub Desktop

- 1 Download GitHub Desktop from desktop.github.com, install it and click **Sign in to GitHub.com**. Approve it in your browser. If it asks you to configure Git, keep its suggestions and click **Finish**.
- 2 Choose **File** → **Clone repository...** → the **GitHub.com** tab → select `yourname/surf-frame`.
- 3 Set **Local path** so the folder lands in your home folder:
 - Mac: `/Users/YOURNAME/surf-frame`
 - Windows: `C:\Users\YOURNAME\surf-frame`
- 4 Click **Clone**.

STOP POINT Your private copy is on GitHub and on your computer.

4 Try it on your computer

15 min

By the end of this Part you'll see the frame's pictures and the studio running on your computer, using made-up sample data. Nothing goes online.

4.1 Go into the project folder

The terminal always works “inside” one folder. `cd` means “change directory”.

MAC

```
cd ~/surf-frame
```

WINDOWS

```
cd %USERPROFILE%\surf-frame
```

Check: type `ls` (Mac) or `dir` (Windows). You should see `README.md`, `pipeline`, `web` and others.

4.2 Create a private Python space and install the parts

A “virtual environment” keeps this project's add-ons separate from the rest of your computer.

MAC

```
python3 -m venv .venv  
source .venv/bin/activate
```

WINDOWS

```
python -m venv .venv  
.venv\Scripts\activate.bat
```

Your prompt now starts with `(.venv)`. From here on, `python` works on both. Install the add-ons (2–5 minutes; lots of text scrolling past is normal):

```
pip install -r requirements.txt  
python -m playwright install chromium
```

The second command downloads a private copy of the Chrome browser that draws the pictures.

EVERY NEW TERMINAL

Each time you open a new terminal to work on the project, repeat 4.1 and the **activate** line from 4.2 first.

4.3 Draw the pictures with sample data

```
python -m pipeline.main --sample --out out
```

After 10–30 seconds the last lines say `rendered 14 layouts` and `rendered 1 frame(s)`. Open the pictures:

MAC

```
open out/renders
```

WINDOWS

```
start out\renders
```

Double-click `classic-grey.png`. That is what the frame will show (with made-up numbers).

4.4 Try the studio

```
python -m pipeline.devserver --out out --sample
```

- 1 Open **`http://127.0.0.1:8787`** in your browser. You'll see a card for each layout.
- 2 Click **Edit** on one. Drag a box to move it, drag its bottom-right corner to resize, click a widget on the left to add it, press **Delete** to remove the selected one.
- 3 Click **Save**, then **Render now** at the top, and go back to the layouts to see the new picture.
- 4 When you've finished, click the terminal and press **Ctrl** + **C** to stop it.

This laptop studio only changes the `out` folder on your computer. The real one goes online in Part 7 and has its own copy of the layouts.

STOP POINT Everything works on your computer.

5 Make it yours: beaches and settings

30 min

All the personal choices live in one file, `config/settings.toml`. Open Visual Studio Code → **File** → **Open Folder...** → choose your `surf-frame` folder (click **Yes, I trust the authors** if asked). Then open **config** → **settings.toml** in the list on the left.

Lines starting with `#` are notes and are ignored. Keep the quotes around text values.

5.1 Your beaches

The example has two beaches, Saunton and Rest Bay. The simplest way to use your own is to **keep the two section labels** in square brackets (`[spots.saunton]` and `[spots.rest_bay]`; they're internal names you never see on the frame) and change the values underneath. Then every starter layout works straight away.

```
[spots.saunton]          # leave this label as it is
name = "Fistral"         # shown on the frame
surflines_id = "..."    # ignore (Surflines is switched off)
lat = 50.4172            # the beach (Google Maps: right-click → copy the numbers)
lon = -5.1036
facing_deg = 300         # direction the beach faces, looking out to sea
admiralty_station = ""   # fill in during Part 9
rules = "saunton"        # leave (board rules, 5.4)
tide_max_m = 7.5         # about the highest spring tide there, in metres
```

SETTING	HOW TO FILL IT IN
lat / lon	Right-click the beach in Google Maps; the two numbers at the top of the menu are latitude then longitude. Click them to copy.
facing_deg	The compass direction you look in when you face the sea: north 0, east 90, south 180, west 270 (north-west about 315). It decides offshore/onshore wind, so it's worth getting within about 20°.
tide_max_m	Sets the top of the tide chart, so small and big tides look different. Use the biggest high tide in a tide table for the nearest port. A rough figure is fine.

DELETE SAUNTON'S HEIGHT FORMULA

Under the Saunton section there's a block starting `[spots.saunton.surf_model]` (with `a`, `b`, `c` and `min_ratio` lines). It was tuned for Saunton's beach. Unless your beach *is* Saunton, delete that block from the `[spots.saunton.surf_model]` line down to the `min_ratio` line. Without it, the general formula is used.

Only one beach? Delete the whole `[spots.rest_bay]` section. Widgets that pointed at it will show your first beach. **More than two?** Copy a section and give it a new label, e.g. `[spots.croyde]`, then pick it per widget in the studio.

Surf heights from the free Open-Meteo forecast are an estimate made from the swell, period and direction. They're usually within about a foot. Appendix A explains how to fine-tune them for your beach.

5.2 Your home town and units

Under `[home]`, set `name`, `lat` and `lon` to your town (town-level is plenty; there's no need for your street). The weather widgets can show it. Under `[units]`, `wave` is `"ft"` or `"m"` and `wind` is `"mph"`, `"kph"` or `"kts"`.

5.3 Wetsuits

The `[wetsuit]` section picks between a winter suit (4.5/3.5 with hood) and a summer suit (3/2) from the water temperature, air temperature and wind. Put your own suits' names in `hooded_name` and `summer_name`, keeping the thickness at the start, e.g. `"4.5/3.5 0'Neill Hyperfreak hooded"`. Adjust the temperatures if you run warmer or colder.

5.4 Boards and fins (optional)

The "Board & fins" widget picks a board and fin set-up from the forecast using `config/board_fin_rules.json`, which holds an **example quiver** of two boards and some fin set-ups. To make it yours, change only the text after `"name"`, `"short"` and `"label"`, and the `min_ft / max_ft` size bands. Keep the keys before the colons as they are: the code refers to them. If you'd rather not bother, just leave that widget out of your layouts.

5.5 The wave map area

The `[map]` section sets the area of the wave map. The default covers the UK, Ireland and the Atlantic to the west. Leave it unless you're outside the UK (Appendix D).

5.6 Check and save to GitHub

- 1 Save the file (`Cmd + S` / `Ctrl + S`).
- 2 In the terminal (folder and activate line from Part 4), run the sample again and look at the pictures. Your beach names should appear:

```
python -m pipeline.main --sample --out out
```

If it stops with an error mentioning `settings.toml`, a quote or a bracket is missing near the line it names.

- 3 Open GitHub Desktop. It lists the changed files. Bottom left, type a summary such as `My beaches`, click **Commit to main**, then **Push origin** at the top.

STOP POINT The frame knows your beaches. Next you'll set up the cloud side.

6 Cloudflare account and storage

20 min

Cloudflare will hold the pictures and run the two small websites. You'll create one private storage bucket and one key that can only touch that bucket. Cloudflare renames menus now and then; if a name differs slightly, look for the closest match.

6.1 Create the account and lock it down

- 1 Go to **dash.cloudflare.com/sign-up**, create a free account and verify your email. If it asks you to add a domain, skip it: you don't need one.
- 2 Turn on two-factor sign-in: person icon (top right) → **Profile** → **Authentication** → **Two-Factor Authentication**. Save the backup codes in your password manager.

6.2 Switch on R2 storage

- 1 In the left menu choose **Storage & databases** → **R2** (sometimes "R2 Object Storage").
- 2 Cloudflare asks for an R2 subscription and a payment card, even on the free plan. This project stores well under 1 MB, far inside the free 10 GB, so you won't be charged. Complete the checkout.
- 3 On the R2 overview page, find **Account ID** on the right (under Account Details). Copy it into your password manager as **Cloudflare Account ID**.

6.3 Create the bucket

- 1 Click **Create bucket**. Name: `surf-frame` (exactly this, lower case).
- 2 Location: **Provide a location hint** → **Western Europe (WEUR)**. Leave any **jurisdiction** option unticked (it changes the web address the code expects).
- 3 Click **Create bucket**. Then open it → **Settings** and check public access is **disabled**, with no `r2.dev` URL or custom domain. The bucket must stay private.

6.4 Create a key that only works on this bucket

- 1 Back on the R2 overview page, click **Manage** next to **API Tokens** (right-hand side) → **Create Account API token**.
- 2 Token name `surf-frame-github`; Permissions **Object Read & Write; Apply to specific buckets only** → **surf-frame**; TTL **Forever** (or a year, with a calendar reminder).
- 3 Click **Create API Token**. The next page shows values **once only**. Copy **Access Key ID** and **Secret Access Key** into your password manager now.

Closed the page too early? Delete that token and create another.

STOP POINT You have a private bucket and a key limited to it.

7 Put the two websites online

20 min

You'll upload the frame server and the studio, and create the frame's secret token. Both stay locked until later Parts, so nothing is exposed in the meantime.

7.1 Connect your terminal to Cloudflare

MAC

```
cd ~/surf-frame/workers/frame-img
```

WINDOWS

```
cd %USERPROFILE%\surf-frame\workers\frame-img
```

```
npx wrangler login
```

If it asks `Ok to proceed? (y)`, type `y` and press Enter. Your browser opens a Cloudflare page: click **Allow**, then return to the terminal. It should say `Successfully logged in`.

7.2 Put the frame server online

```
npx wrangler deploy
```

If it asks you to register a **workers.dev subdomain**, type a short name that doesn't reveal who you are (e.g. `tidewatch42`). It becomes part of both web addresses. At the end it prints an address like `https://surf-frame-img.tidewatch42.workers.dev`. Save it as **Frame server address**.

7.3 Create the frame's secret token

This long random string is the frame's password (Windows: `python` instead of `python3`):

```
python3 -c "import secrets; print(secrets.token_urlsafe(32))"
```

Copy the ~43 characters it prints into your password manager as **Device token**. Then give it to the website:

```
npx wrangler secret put DEVICE_TOKEN
```

At Enter a secret value: paste the token and press Enter. Nothing appears as you paste; that's normal. It ends with `Success`.

7.4 Put the studio online

MAC

```
cd ~/surf-frame/workers/studio  
npx wrangler deploy
```

WINDOWS

```
cd %USERPROFILE%\surf-frame\workers\studio  
npx wrangler deploy
```

It prints an address like `https://surf-frame-studio.tidewatch42.workers.dev`. Save it as **Studio address**. Open it: you should see just the word **Forbidden**. That's the safety lock: the studio turns everyone away until Part 8 is done.

STOP POINT Both websites are live and locked.

8 Lock the studio so only you can open it

25 min

Two locks protect the studio. Cloudflare Access makes you sign in before the page loads, and then the studio itself checks the sign-in is genuine and is yours.

8.1 Switch on Cloudflare Zero Trust (free)

- 1 In the Cloudflare dashboard's left menu, click **Zero Trust**.
- 2 First time only: choose a team name (e.g. `tidewatch42`). Your team domain becomes `tidewatch42.cloudflareaccess.com`. Save it as **Zero Trust team domain**.
- 3 Choose the **Free** plan. If it asks for payment details, the Free plan still costs nothing. Sign-in by a one-time code emailed to you is on by default.

8.2 Put the studio behind Access

- 1 Main dashboard → **Workers & Pages** → **surf-frame-studio** → the **Access** tab → **Protect this Worker behind Access**.
- 2 Choose **All traffic** (not "Previews only").
- 3 Authentication policy: **Cloudflare account** (only people in your Cloudflare account, which is just you). **Don't** choose "Email domain" with gmail.com: that would let every Gmail user sign in.
- 4 Click **Apply Access**.

NOT THE FRAME SERVER

Don't do this for **surf-frame-img**. The frame can't sign in; that site is protected by its token instead.

8.3 Copy the application's AUD tag

Go to **Zero Trust** → **Access** → **Applications** (may be under **Access controls**), click the **surf-frame-studio** application, and find **Application Audience (AUD) Tag**: a long string of letters and numbers. Save it as **Access AUD tag**.

8.4 Tell the studio who you are

In Visual Studio Code open **workers** → **studio** → **wrangler.toml**. In the `[vars]` section fill in these values between the quotes:

LINE	PUT IN
ACCESS_TEAM_DOMAIN = ""	your team domain, e.g. "tidewatch42.cloudflareaccess.com"
ACCESS_AUD = ""	the AUD tag from 8.3
ALLOWED_EMAILS = ""	the email address you use to log in to Cloudflare
GH_REPO = ""	your GitHub username, a slash, then surf-frame, e.g. "jo-surfs/surf-frame"
MAIN_FRAME_NAME	optional: what your frame is called in the studio

Save the file, then upload the studio again (the terminal should still be in `workers/studio`):

```
npx wrangler deploy
```

8.5 Test both locks

- 1 Open your Studio address in a private/incognito window. You should get a Cloudflare sign-in page, not the studio.
- 2 Enter your Cloudflare email, then the code emailed to you. The studio opens with an empty gallery and "no forecast data yet". That's correct until Part 10.
- 3 **Forbidden** after signing in? One of the values in 8.4 is wrong. Check them, save and run `npx wrangler deploy` again.

8.6 Save the change to GitHub

In GitHub Desktop: summary `Configure studio` → **Commit to main** → **Push origin**. Your repository is private, so these values stay with you.

STOP POINT Only you can open the studio.

9 UK tide times from ADMIRALTY

20 min

ADMIRALTY (the UK Hydrographic Office) gives official tide times free on its Discovery plan. You'll get a key, find the tide stations nearest your beaches, and do a first test with live data. Without this the frame still shows tides, estimated from the Open-Meteo forecast, but they're less accurate and the heights aren't chart datum.

9.1 Get your key

- 1 Go to **developer.admiralty.co.uk** → **Sign up**, and confirm your email.
- 2 **Products** → **UK Tidal API - Discovery** → **Subscribe**. Give it any name (e.g. `surf-frame`), accept the terms and confirm.
- 3 On your **Profile** page, under Subscriptions, click **Show** next to **Primary key**. Save it as **ADMIRALTY subscription key**.
- 4 Put a reminder in your calendar for 11 months from now: the free Discovery subscription lasts a year and needs renewing.

9.2 Find the tide stations

In a terminal, go to the project and activate it (Part 4.1 and the activate line from 4.2). Give this window your key (it's forgotten when you close the window):

MAC

```
export ADMIRALTY_KEY=YOURKEY
```

WINDOWS

```
set ADMIRALTY_KEY=YOURKEY
```

```
python -m pipeline.find_station
```

It lists the five stations nearest each beach, each with a 4-digit number (e.g. `0512`). Pick the nearest one on the open coast, not up an estuary. You can also search by name: `python -m pipeline.find_station newquay`.

9.3 Save the stations

In `config/settings.toml`, set `admiralty_station = "0000"` under each beach to its number (keep the quotes and any leading zero). Save.

9.4 First test with real data

```
python -m pipeline.main --out out
```

Near the end you'll see a line per beach like:

```
[saunton] surf source=open-meteo tides=admiralty warnings=[]
```

`tides=admiralty` means your key and stations work. `tides=open-meteo` means the station number or the key is wrong. Open `out/renders/classic-grey.png` to see your real forecast.

On a Mac you'll also see a line saying the wave map failed. That's expected: the map's file reader isn't installed on Macs, and the map works in the cloud.

Commit and push in GitHub Desktop (`Add tide stations`).

STOP POINT Real forecasts and tides work on your computer.

10 First cloud render

20 min

GitHub keeps your keys in an encrypted store called secrets. Nobody can read them back, not even you; the job can only use them.

10.1 Create a protected place for the secrets

- 1 On github.com open your **surf-frame** repository → **Settings** → **Environments** → **New environment**.
- 2 Name: `production` (exactly). Click **Configure environment**.
- 3 Under **Deployment branches and tags**, choose **Selected branches and tags** → **Add deployment branch or tag rule** → type `main` → **Add rule**.

10.2 Add the secrets

On the same page, under **Environment secrets**, click **Add environment secret** once per row. Type the name exactly (capitals and underscores) and paste the value from your password manager.

NAME	VALUE
<code>R2_ACCOUNT_ID</code>	Cloudflare Account ID
<code>R2_ACCESS_KEY_ID</code>	R2 Access Key ID
<code>R2_SECRET_ACCESS_KEY</code>	R2 Secret Access Key
<code>R2_BUCKET</code>	<code>surf-frame</code>
<code>ADMIRALTY_KEY</code>	ADMIRALTY subscription key

A typo can't be seen afterwards, only replaced: click the pencil next to a secret to paste it again.

10.3 Flip the switch and run it

- 1 Still in **Settings: Secrets and variables** → **Actions** → the **Variables** tab → **New repository variable**. Name `SURF_FRAME_READY`, value `true` → **Add variable**. This switches the automatic jobs on.
- 2 Click the **Actions** tab. If asked, click **I understand my workflows, go ahead and enable them**.
- 3 Click **render** in the left list → **Run workflow** (right) → keep Branch: `main` → the green **Run workflow**.
- 4 Refresh after a few seconds. A run appears with a yellow dot and takes 3–5 minutes. A green tick means it worked.

10.4 Check the results

- Open your Studio address and sign in. You should see the layouts with real pictures, with **Classic** marked as on the frame. The starter layouts' names mention Saunton or Rest Bay; rename them in the editor (click the name at the top).
- The grey bar at the top lists each beach's data source. Orange text means that beach used a backup source; hover to see why.
- Build your frame link from the Frame server address and Device token, and open it in a browser:

```
https://surf-frame-img.YOURSUBDOMAIN.workers.dev/frame/YOURDEVICETOKEN.png
```

You should see the frame's picture. Treat that link like a password.

RED CROSS?

Click the failed run → **render** → the step with the red cross, and read the last few lines. The common causes are in Appendix B.

From now on the job runs by itself every 3 hours at 7 minutes past (UTC): about 01:07, 04:07, 07:07 ... in UK summer time. GitHub's own timer often starts late, which Part 11 fixes.

STOP POINT The forecast pictures now update themselves.

11 On-time updates and hands-off deploys

25 min

Two small upgrades. The first lets Cloudflare start each render exactly on time and adds a **Render now** button to the studio. The second makes website changes deploy themselves when you change the code on GitHub.

11.1 On-time renders and the Render now button

The studio starts a render every 3 hours on Cloudflare's timer, which is punctual, instead of relying on GitHub's, which often runs late. For that it needs a GitHub token that can only start this one job.

- 1 On github.com: profile picture → **Settings** → **Developer settings** (bottom of the left menu) → **Personal access tokens** → **Fine-grained tokens** → **Generate new token**.
- 2 Name `surf-frame-render`, Expiration **90 days**. Repository access: **Only select repositories** → **surf-frame**.
- 3 Permissions → Repository permissions → **Actions: Read and write**. Leave everything else as No access. Click **Generate token** and save it as **GitHub render token** (shown once).

- 4 In a terminal, go to the studio folder and store it:

MAC

```
cd ~/surf-frame/workers/studio
```

WINDOWS

```
cd %USERPROFILE%\surf-frame\workers\studio
```

```
npx wrangler secret put GH_TOKEN
```

- 5 Reload the studio: a **Render now** button appears top right. Press it and new pictures arrive in about 3 minutes.
- 6 Put a reminder in your calendar for 85 days' time to make a new token and repeat step 4. If it expires, renders carry on using GitHub's slower timer.

11.2 Website updates deploy themselves

Whenever you change the websites' code on GitHub (or the layout editor itself), a workflow tests it and uploads both websites. It needs a Cloudflare token that can only update Workers, kept in its own environment, so the 3-hourly job never sees it.

- 1 Cloudflare dashboard → person icon → **Profile** → **API Tokens** → **Create Token** → next to **Edit Cloudflare Workers** click **Use template**.
- 2 Account Resources: **Include** → **your account**. Zone Resources: **Include** → **All zones** (you have none, so this grants nothing extra). TTL: an end date a year from today.
- 3 **Continue to summary** → **Create Token**. Save it as **Cloudflare deploy token**.
- 4 GitHub repository → **Settings** → **Environments** → **New environment** `cloudflare-deploy`, restricted to `main` as in 10.1. Add two environment secrets:

NAME	VALUE
<code>CLOUDFLARE_API_TOKEN</code>	Cloudflare deploy token
<code>CLOUDFLARE_ACCOUNT_ID</code>	Cloudflare Account ID

- 5 **Actions** → **deploy-websites** → **Run workflow**. After 3–5 minutes you should see a green tick with three boxes: test, deploy (frame-img), deploy (studio). Check the studio still opens and your frame link still shows the picture. Deploying keeps your secrets and the sign-in lock.

CHANGING THINGS LATER

Edit a file (or use GitHub's website editor), commit and push. If you change `config/`, `pipeline/`, `layouts/` or `web/`, a fresh render runs automatically; if you change `workers/` or `web/`, both websites redeploy. The **Actions** tab shows each run.

STOP POINT The cloud side is finished. Next, the frame.

12 Set up the frame

60 min + a day's bench test

The kit runs TRMNL's own open-source firmware, pointed at your frame server instead of `trmnl.app`. Every three hours it wakes, asks your server what to show, downloads the picture, draws it and goes back to sleep.

12.1 Give the frame its own Wi-Fi

The firmware uses encrypted connections but doesn't check the server's certificate, so keep the frame away from your other devices:

- 1 In your router's settings, turn on a **guest** (or IoT) network on **2.4 GHz**. The board has no 5 GHz.
- 2 Security **WPA2** (or WPA2/WPA3 mixed) with its own strong password. Save both in your password manager.
- 3 Turn on **client isolation** (often "allow guests to see each other" = off) and turn off access to the local network.

A combined 2.4 + 5 GHz network name is usually fine. If the frame won't join, make a 2.4 GHz-only guest network.

12.2 Watch the server's log

Keep this running in a terminal for the rest of this Part. It's where the frame's MAC address, battery and signal show up:

MAC

```
cd ~/surf-frame/workers/frame-img
npx wrangler tail surf-frame-img --format pretty
```

WINDOWS

```
cd %USERPROFILE%\surf-frame\workers\frame-img
npx wrangler tail surf-frame-img --format pretty
```

The Cloudflare dashboard has the same log, with history: Workers & Pages → surf-frame-img → Logs.

12.3 Put the parts together on a table

The screen's ribbon cable is fragile: don't fold it, and don't lift the screen by it.

- 1 **Power switch OFF.**
- 2 **Antenna:** press its small round plug straight onto the board's antenna socket until it clicks.
- 3 **Screen ribbon:** flip the board's connector latch up gently, slide the ribbon in with the **metal contacts facing up**, all the way and square, and close the latch. Test without the extension cable first.
- 4 **Battery:** plug its lead in (it only fits one way) and check the lead isn't pinched. Leave the switch off for now; USB power is enough for flashing.

12.4 Flash the TRMNL firmware

- 1 Plug the board into your computer with the **data** cable.
- 2 Open **trmnl.com/flash** in Chrome or Edge (Firefox can't do this).
- 3 Choose the **Seeed 7.5" OG DIY Kit** build (the label may differ slightly) and take the newest version offered.
- 4 Click **Connect** and pick the **USB JTAG/serial debug unit**, then **Install**. Allow erasing if asked. It takes 1–2 minutes; don't unplug.
- 5 Board not listed, or install fails? Unplug, **hold BOOT**, plug back in while holding it, release, and Connect again. Also try another cable or USB port.
- 6 When it finishes, the screen flashes and shows TRMNL's Wi-Fi setup screen. That proves the screen, ribbon and board all work. Blank screen? Power off, re-seat the ribbon (metal up, fully in, latch closed) and press RESET.

12.5 Connect it to Wi-Fi and your server

- 1 On your phone, join the Wi-Fi network **TRMNL** (no password). The setup page opens by itself; if not, browse to **http://4.3.2.1**.
- 2 Choose your **guest network** and enter its password.
- 3 Tap **Advanced** → **Custom Server** and enter your Frame server address exactly, e.g. `https://surf-frame-img.tidewatch42.workers.dev`: starting with `https://`, with **no slash at the end**.
- 4 Save / Connect. The frame joins Wi-Fi and calls your server. It's **refused** this first time, because the server doesn't know it yet: the screen says it isn't registered, and the log window shows a line like:

```
{"event":"setup_refused","mac":"A1B2C3D4E5F6","model":"...","fw":"..."}
```

12.6 Register the frame and pair it

- 1 Copy the `mac` value from that line into your password manager. In a **second** terminal, go to `workers/frame-img` (as in 12.2) and run:

```
npx wrangler secret put DEVICE_MACS
```

Paste the MAC and press Enter. It works immediately.

- 2 Press **RESET** on the board (or **KEY3**, the button next to it). If you do nothing, it retries within about 5 minutes.
- 3 The log now shows `setup_ok` then `display`, and the screen shows your forecast. A 4-grey update flashes black and white for a few seconds first; that's normal.

12.7 Leave it on the bench for a day

- ☐ Switch the **power switch ON** and unplug USB. It should keep updating on battery.
- ☐ It wakes about 25 minutes after each render: roughly 01:32, 04:32, 07:32, 10:32, 13:32, 16:32, 19:32 and 22:32 in UK summer time (an hour earlier in winter).
- ☐ In the log, `battery_v` is about 4.1–4.2 when full; recharge over USB-C at about 3.5. Expect several weeks per charge.
- ☐ `rsi` (signal) better than about -70 is comfortable; worse than -80 means moving the frame or the antenna.
- ☐ Press **KEY3**: it should switch to the wave map within a few seconds (Part 14). Press it again to switch back.

12.8 The buttons

BUTTON	PRESS	DOES
RESET	click	Restart, then fetch and draw
KEY3	click	Wake and switch between your layout and the button layout (Part 14)
KEY3	hold 5 s	Forget Wi-Fi and start the TRMNL setup hotspot again (to change network or server address)
KEY3	hold 15 s	Wipe all its settings and start setup again (the MAC stays registered, so it pairs straight away)
BOOT	hold while plugging in USB	Flashing mode (only for re-flashing)

STOP POINT The frame shows your forecast and updates on battery.

13 Build it into a picture frame

1–2 hours

- 1 **Screen into the mount:** lay the mount face down on a soft cloth. Place the screen face down, ribbon edge at the bottom or side you planned, and centre the **picture area** (not the glass) in the window. Power it up once to check the picture is square before fixing anything.
- 2 **Fix the screen** with thin tape at the edges of the glass, not glue. Don't press on the screen face or bend it.

- 3 **Ribbon to the back:** cut a slot in the backing board and feed the ribbon through without a sharp fold. If it won't reach comfortably, use the extension cable and adapter (metal contacts **up** at each connector). Re-test the screen before closing up.
- 4 **Board** on the back of the backing board with foam pads. Keep **USB-C** reachable for charging, and the **power switch, RESET and KEY3** reachable (or at least reachable by opening the back).
- 5 **Battery:** foam tape, flat, not squeezed by the frame back, away from screw points and anything warm. Never pierce, crush or bend it.
- 6 **Antenna:** stick it to the backing board as high and as far from the battery and any metal as you can. Poor signal? Bring it outside the frame back.
- 7 Close up, stand it up, press KEY3 twice and check the picture.

MOUNT COVERS THE EDGE?

Studio → **Frame settings** → **Edge margin:** 1–3 px (or more) of white border on every side. The whole picture is drawn smaller, so text stays sharp.

14 Using the studio and the button

Open your Studio address on any computer or phone and sign in with the emailed code. Every layout is re-drawn with fresh data every 3 hours, so switching what the frame shows is instant.

14.1 The gallery

BUTTON	WHAT IT DOES
Show on frame	Makes this layout the one the frame downloads at its next wake (press KEY3 to see it now). No reflashing.
Edit	Opens the layout editor
Duplicate	Copies a layout so you can experiment without losing the original
Delete	Removes a layout (not the one on the frame)
New layout	Starts a blank layout with just the date header
Render now	New pictures in about 3 minutes (Part 11)
Screen size	Shows the frame at its real size on your screen

14.2 The editor

- **Add a widget:** pick its detail level (Minimal, Standard or Full) and click its name on the left. It's previewed, then placed in the first free space.
- **Move** by dragging; **resize** with the handle at the bottom-right corner. Text grows and shrinks to fit, and less important parts drop out of small boxes (never cut off with "...").
- **Change it:** with a widget selected, the right-hand panel shows its options: which beach, how many hours or days, border style and so on.
- **Keyboard:** arrows move, **Shift** + arrows resize, **Ctrl** + **D** duplicates, **Delete** removes, **Ctrl** + **Z** undoes.
- **Layout settings:** click an empty part of the frame to choose the default beach, **4 greys** or **black & white**, and invert.

- **Save** keeps your changes; **Save & show on frame** also puts it on the frame. Edits reach the frame at the next render, or after Render now.

The widgets include surf height and rating, swell, wind, wind by hour, tide times and tide chart (24/48/72 h), a surf & wind chart, first and last light, water temperature and wetsuit, board & fins, best surf windows, a multi-day outlook, weather (now, by hour, week), moon, a text note, a flight countdown and the wave map.

14.3 The wave map and the KEY3 button

The **Wave map** layout shows significant wave height (the average height of the bigger waves out at sea, in metres) from NOAA's global wave model for the next 3-hourly time after each render. The four shades adapt to the day, e.g. 0–1, 1–2, 2–3 and 3+ m. Arrows show which way the waves are travelling, and dots mark your beaches.

Press KEY3 on the frame to switch from your normal layout to the wave map; press again to switch back. At the next scheduled update it always goes back to your normal layout. To use a different layout for the button, or turn it off: **Frame settings** → **Button (KEY3)** → **Button shows**.

14.4 Frame settings

- **Edge margin**: white border for a tight mount (Part 13).
- **Button (KEY3)**: what the button switches to.
- **Home town**: overrides [home] in the settings file for the weather widgets.

A Optional extras

Flight countdown

The **Flight** widget shows a flight's airports, date, departure and arrival times and days to go, and can appear automatically a set number of days before. Type the details into the widget's boxes yourself, or let it look them up from the flight number:

- 1 Sign up at **rapidapi.com**, search for **AeroDataBox**, and subscribe to its free **Basic** plan (check the monthly limit shown).
- 2 On its **Endpoints** page, copy the value of `X-RapidAPI-Key` from the code sample.
- 3 GitHub → Settings → Environments → **production** → Add environment secret `AERODATABOX_KEY` with that value.

Far-off flights may only be found nearer the date. Details you type in always win.

Friends' frames

Friends can run their own frame on your setup: same forecast, their own layouts, chosen by them in your studio. They only buy the kit, flash it and type in an address you send them.

- 1 Once: make 48 random characters (`python3 -c "import secrets; print(secrets.token_hex(24))"`) and store the **same** value on both websites: `npx wrangler secret put FRAME_SECRET` in `workers/frame-img` and again in `workers/studio` .
- 2 Change the studio's Access policy (Part 8.2) from "Cloudflare account" to **Emails**, listing your email and each friend's.
- 3 Studio → **Friends' frames** → add their name and login email → copy their server address (`.../f/<id>/<code>`) and send it with your studio link. Their frame pairs itself; no MAC needed.

Friends never see your home town, bookings, board picks or wetsuit names. Removing a frame deletes its files; also take their email off the Access policy.

The Wave (Bristol) booking widget

If you surf at The Wave, the frame can show your next session. Your normal Gmail forwards The Wave's confirmation emails to a new inbox used only for this, which the job reads. Only the date, time and session type are kept.

- 1 Create a new Gmail account (in a private window), turn on **2-Step Verification**, then create an **app password** at **myaccount.google.com/apppasswords** (16 letters, shown once).
- 2 In your own Gmail: gear → **See all settings** → **Forwarding and POP/IMAP** → **Add a forwarding address** → the new inbox, and confirm from the email it receives. Leave forwarding itself **disabled**.
- 3 Create a Gmail filter: **From** `thewave.com` (check the sender of a real confirmation) → **Forward it to** the new inbox.
- 4 Test the reader on a downloaded confirmation (`:` → Download message):
`python -m pipeline.bookings --parse PATH_TO_FILE.eml` .
- 5 Add `GMAIL_USER` and `GMAIL_APP_PASSWORD` to the **production** environment, set `enabled = true` under `[bookings]` in `settings.toml`, commit and push.

Emails must come from an allowed domain and carry a valid DKIM signature, so a fake email can't put a session on the frame. Gmail only forwards mail that arrives after the filter exists, so your next booking is the first to appear.

Fine-tuning surf heights for your beach

Note the surf height you see (or a forecast you trust) for a few weeks in `calibration/<spot>-YYYY-MM.csv` files, then run `python -m pipeline.calibrate <spot>`. It fits a height formula for that beach and prints the `surf_model` lines for `settings.toml`. `calibration/README.md` explains the format.

Automatic package updates

Rename `.github/dependabot.example.yml` to `dependabot.yml` to get a weekly pull request when a building block has an update. Merge it if the checks are green.

Surfline

Surfline has no public API. The code can use the endpoints its website uses, but it's off by default (`[sources] surfline = false`): check Surfline's terms of use, which restrict automated access, before changing that. It has also been refusing requests from GitHub's servers since September 2026.

B Troubleshooting

On your computer

YOU SEE	WHAT TO DO
<code>python3: command not found / 'python' is not recognized</code>	Close and reopen the terminal. Windows: reinstall Python with "Add python.exe to PATH" ticked (2.2).
<code>'npx' is not recognized</code>	Close and reopen the terminal after installing Node.js (2.3).
No module named <code>pipeline</code>	Wrong folder: run the <code>cd</code> line from 4.1.
No module named <code>...</code> / <code>externally-managed-environment</code>	The Python space isn't on: run the activate line from 4.2. Your prompt should start with <code>(.venv)</code> .
Errors while <code>pip install</code> runs	Check <code>python --version</code> inside the Python space says 3.12 or newer. If not, install a newer Python (2.2), delete the <code>.venv</code> folder and redo 4.2.
An error naming <code>settings.toml</code>	A missing quote, bracket or <code>=</code> near the line it names. Compare with the example lines.
Wrangler: Authentication error / not logged in	<code>npx wrangler login</code> again (7.1).

In the cloud

YOU SEE	WHAT TO DO
Studio says Forbidden after you sign in	A value in 8.4 is wrong, or you signed in with a different email than <code>ALLOWED_EMAILS</code> . Fix, save, <code>npx wrangler deploy</code> .
No sign-in code email	Check spam. Use the email you log in to Cloudflare with.
Render: <code>InvalidAccessKeyId</code> , <code>SignatureDoesNotMatch</code> , <code>AccessDenied</code>	Re-paste the two R2 key secrets (10.2). Still failing: make a new R2 token (6.4).
Render: <code>NoSuchBucket</code>	<code>R2_BUCKET</code> must be exactly <code>surf-frame</code> .
Render: <code>KeyError: 'R2_...'</code>	A secret name is misspelt, or it was added to the repository instead of the production environment.

Render and deploy runs are skipped	The <code>SURF_FRAME_READY</code> variable is missing or isn't exactly <code>true</code> (10.3).
<code>no surf data from any source</code>	The forecast service didn't answer. It'll try again in 3 hours; the frame keeps its last picture.
Frame link says Not found	Check the token part exactly and that it ends in <code>.png</code> , and that the latest render has a green tick.
Wave map says "No wave map yet"	NOAA's data wasn't reachable. The last good map is reused for up to 6 hours, then it waits for the next render.

On the frame

YOU SEE	WHAT TO DO
Computer doesn't see the board	Charge-only cable, or not in flashing mode: try another cable; hold BOOT while plugging in.
Screen stays blank after flashing	Ribbon upside down or not fully in: metal side up, fully in, latch closed, RESET.
TRMNL hotspot never appears	It already has Wi-Fi saved: hold KEY3 for 5 s.
Joins Wi-Fi but nothing in the log	Server address wrong (slash at the end, typo, <code>http://</code>) or the guest network blocks the internet. Re-enter via KEY3 5 s.
"Not registered" after adding the MAC	Press RESET or KEY3. Check <code>DEVICE_MACS</code> matches the log exactly.
Log shows <code>display_bad_token</code>	You changed <code>DEVICE_TOKEN</code> . It re-pairs by itself on the next wake, or press RESET.
KEY3 refreshes but doesn't switch to the map	Check Frame settings → Button shows isn't "Nothing" and that the Wave map layout exists and has a picture. The log line for that wake shows <code>"source"</code> (should be <code>EXT1</code> , <code>EXT0</code> or <code>button</code>) and <code>"button_layout":true</code> .
Picture never changes	Check the latest render has a green tick, then press KEY3 twice (map and back).
Only black and white, no greys	That layout is set to black & white: choose 4 greys in its layout settings and Render now.
Faint ghost of the old picture	Normal on e-ink between full refreshes; 4-grey updates clear it.

C Upkeep, costs and security

Regular upkeep

WHEN	TASK
Every few weeks	Charge the frame over USB-C when <code>battery_v</code> nears 3.5.
Every 85 days	New GitHub render token → <code>npx wrangler secret put GH_TOKEN</code> in <code>workers/studio</code> (11.1).
After 11 months	Renew the ADMIRALTY Discovery subscription; if the key changes, update <code>ADMIRALTY_KEY</code> (10.2). Renew the Cloudflare deploy token (11.2).
Now and then	Check GitHub and Cloudflare still have two-factor on, and delete tokens you no longer use.

GitHub emails you if a run fails, so you'll know if anything needs attention.

What it costs

Nothing beyond the hardware, as long as usage stays inside the free allowances (true at the time of writing; check the providers' pricing pages if in doubt):

SERVICE	THIS PROJECT USES	FREE ALLOWANCE
GitHub Actions (private repo)	about 600–700 minutes a month	2,000 minutes a month
Cloudflare R2	a few MB, a few hundred reads a day	10 GB, millions of reads
Cloudflare Workers	a few dozen requests a day	100,000 requests a day
Cloudflare Zero Trust	1 user	up to 50 users
ADMIRALTY Discovery	about 16 calls a day	10,000 calls a month, 1 year
Open-Meteo, NOAA	a few calls per render	free for non-commercial use

Security checklist

- ☐ Two-factor sign-in on GitHub and Cloudflare.
- ☐ Your `surf-frame` repository is **private**.
- ☐ The R2 bucket has public access off, and its key only works on that bucket.
- ☐ The studio is behind Cloudflare Access, and `ALLOWED_EMAILS` lists only you (and friends you've added).
- ☐ The frame is on a guest network with client isolation.
- ☐ `DEVICE_MACS` lists only your frame, and the device token is only in Cloudflare and your password manager.
- ☐ The server never sends firmware updates: the frame only changes firmware when you re-flash it over USB.

IF A KEY LEAKS

Cancel it where it came from (R2 token, GitHub token, ADMIRALTY key, Cloudflare token, Gmail app password), make a new one and replace the GitHub or Wrangler secret. For the device token:

```
npx wrangler secret put DEVICE_TOKEN
```

with a new value in `workers/frame-img`. The frame re-pairs by itself, and the old frame link stops working immediately.

D Living outside the UK?

The project was built and tested in the UK. Most of it works anywhere, but these parts need attention (untested elsewhere):

- **Time zone:** set `timezone` at the top of `settings.toml` (e.g. `"Europe/Lisbon"`, `"America/Los_Angeles"`).
- **Tides:** ADMIRALTY covers UK stations only. Leave `admiralty_station = ""`; tides then come from Open-Meteo's sea-level forecast, available worldwide but less exact, with heights relative to mean sea level.
- **Wave map:** change the `[map]` box (west, east, south, north in degrees). The bundled coastline covers 40°W–30°E and 35°N–67°N (north-west Europe and the eastern Atlantic); elsewhere the map shows the waves without a coastline.
- **Units:** metres or feet for waves, and mph, kph or knots for wind, in `[units]`.
- **Update times:** the frame wakes every 3 hours on UTC time, wherever you are.

CREDITS

Forecast data: Open-Meteo.com (CC BY 4.0). Wave map: NOAA GFS-Wave (public domain). Tide times: ADMIRALTY UK Tidal API. Coastline: Natural Earth (public domain). Fonts: Inter and Barlow Condensed (SIL Open Font License). Firmware: TRMNL (open source). The project's code is MIT-licensed: use it, change it and share it, keeping the copyright notice.